

1. Understand Default Methods in Interfaces (Java 8)

What is a Default Method?

A **default method** is a method inside an interface that already has an implementation. It is declared using the default keyword.

Introduced in **Java 8**, it allows interfaces to evolve by adding new methods **without breaking existing implementing classes**.

```
interface Vehicle {  
  
    default void start() {  
        System.out.println("Vehicle Started");  
    }  
}  
  
class Car implements Vehicle { }  
  
public class Main {  
    public static void main(String[] args) {  
        Car car = new Car();  
        car.start();  
    }  
}
```

Output

```
Vehicle Started
```

Why Were Default Methods Introduced?

Suppose thousands of classes implement an interface.

If you add a new abstract method to that interface, **every implementing class breaks** because it must implement the new method.

Default methods solve this by providing a **default implementation**, so existing classes continue working.

Overriding a Default Method

Implementing classes can override it if they need custom behavior.

```
interface Vehicle {  
  
    default void start() {  
        System.out.println("Vehicle Started");  
    }  
}  
  
class Car implements Vehicle {  
  
    @Override  
    public void start() {  
        System.out.println("Car Started");  
    }  
}
```

Important Interview Points

- Introduced in **Java 8** to maintain **backward compatibility**.
 - Declared using the **default** keyword inside an interface.
 - Contains both **declaration and implementation**.
 - **Inherited automatically** by implementing classes.
 - Can be **overridden**, but **not required**.
 - Helps add new functionality to interfaces **without breaking existing implementations**.
 - If a class implements two interfaces with the same default method, it **must override** that method to resolve ambiguity.
 - Default methods participate in **runtime polymorphism** because they are instance methods.
-

Tricky Interview Questions

Q1. Why were default methods introduced?

Answer:

To allow adding new methods to existing interfaces **without forcing every implementing class to change**.

Q2. Can a class override a default method?

Answer: Yes.

```
@Override  
public void start() { }
```

Q3. What happens if two interfaces have the same default method?

```
interface A {  
    default void show() {}  
}  
  
interface B {  
    default void show() {}  
}  
  
class Test implements A, B {  
  
    @Override  
    public void show() {  
        System.out.println("Resolved");  
    }  
}
```

Answer:

The implementing class **must override** the method to remove ambiguity.

Q4. Can default methods be private?

Answer: No.

(Default methods are meant to be inherited.)

Q5. Can default methods be static?

Answer: No.

A method is either default **or** static, not both.

Q6. Can an abstract class override a default method?

Answer: Yes.

Interview Summary

- **Default methods are implemented methods inside interfaces.**
- Introduced in **Java 8** for **backward compatibility**.
- Allow interfaces to evolve **without breaking existing code**.
- Implementing classes **inherit** them automatically and **may override** them.
- If multiple interfaces provide the same default method, the implementing class **must override it**.
- Default methods support **runtime polymorphism** because they are instance methods.

Interview One-Liner:

"A default method is an interface method with an implementation, introduced in Java 8 to add new functionality to interfaces without breaking existing implementations."

2. Use final for Class, Method, and Variable Constraints

What is final?

The **final** keyword is used to **restrict modification** in Java.

It can be applied to:

- Variables
- Methods

- Classes
-

1. Final Variable

A final variable **can be assigned only once**.

```
final int AGE = 22;

// AGE = 25;    ❌ Compile Error
```

2. Final Method

A final method **cannot be overridden** by subclasses.

```
class Animal {

    final void eat() {
        System.out.println("Eating");
    }
}

class Dog extends Animal {

    // void eat() {}    ❌ Compile Error
}
```

3. Final Class

A final class **cannot be inherited**.

```
final class MathUtil {

}

// class Test extends MathUtil {}    ❌ Compile Error
```

Why Use final?

- Prevent accidental modification.
 - Protect important business logic.
 - Improve code safety and maintainability.
 - Create immutable constants.
 - Prevent inheritance where customization should not be allowed.
-

Important Interview Points

- final means **restriction** in Java.
 - A **final variable** can be assigned only once.
 - A **final method** is inherited but **cannot be overridden**.
 - A **final class** cannot be extended.
 - A **final reference variable** cannot point to another object, but the object's state may still change.
 - Constructors **cannot** be final because they are never inherited.
 - final and abstract cannot be used together on a class or method because their purposes conflict.
 - String is a **final class**, making it immutable and preventing subclasses from altering its behavior.
-

Tricky Interview Questions

Q1. Is a final object immutable?

```
final List list = new ArrayList<>();  
list.add("Java");
```

Answer: No.

The **reference** cannot change, but the object's contents can.

Q2. Can a final method be overloaded?

Answer: Yes.

final prevents **overriding**, not **overloading**.

Q3. Can constructors be final?

Answer: No.

Constructors are never inherited, so overriding isn't possible.

Q4. Can an abstract method be final?

Answer: No.

- abstract → must be overridden.
- final → cannot be overridden.

They are opposite concepts.

Q5. Can a final class have constructors?

Answer: Yes.

It simply cannot be inherited.

Q6. Why is String final?

Answer:

To ensure **immutability**, improve **security**, and prevent subclasses from changing its behavior.

Q7. Does final improve performance?

Answer:

Not directly. While the JVM **may** optimize some final usages, final is primarily for **design correctness and preventing modification**, not performance.

Interview Summary

- **final is used to restrict modification in Java.**
- **Final variable** → Cannot be reassigned.
- **Final method** → Cannot be overridden.
- **Final class** → Cannot be inherited.
- A **final reference** is fixed, but the referenced object's state may still change.

- final helps create **constants**, protect business logic, and design **safe, immutable APIs**.
- final is incompatible with abstract because one requires overriding while the other forbids it.

Interview One-Liner:

"final is a restriction keyword in Java: variables cannot be reassigned, methods cannot be overridden, and classes cannot be inherited."